

Team name: DealMakers

Group members (Name, NetID): Alice Lee cl2852, Pin-Hsuan Lai pl662, Pin-Yeh Lai pl663

Part 1: Demand estimation and price optimization (without competition)

The goal was to design a personalized, capacity-constrained pricing system by estimating demand and selecting revenue-maximizing prices while managing inventory limits.

Demand Estimation

Initial analysis showed demand varied significantly by Covariate1 and Covariate2, making segmentation essential. Due to this variation, we tested both global models and segmented models. The strongest global model, LightGBM (~2.49M), was outperformed by the 8-segmented XGBoost model (~3.1M expected, ~3.23M actual), proving segmentation was essential for capturing customer variation, outperforming algorithm switches alone.

We therefore split customers into 8 groups using media cuts on the three covariates and trained an XGboost model for each segment, we predicted purchase probabilities for a grid of 100 prices and selected the price with the highest expected revenue. This personalized approach outperformed all alternatives. As overly aggressive hyperparameters led to unrealistic expected revenue and overfitting, a moderate configuration was selected for stable performance.

Price optimization

After estimating demand, we built a pricing rule combining personalized revenue

maximization with inventory awareness. For each customer, a vectorized computation evaluated 100 possible prices to choose the highest revenue price, extending the prior price search.

To prevent early stockouts and reduce leftover inventory, we applied an inventory-pressure multiplier that adjusts the optimal price depending on how much inventory is left. If inventory is too high compared to the time until replenishment, the multiplier decreases price (up to 2% per extra unit). If inventory was low, the multiplier increases price (up to 4% per shortage unit). These adjustments delayed early stockouts and reduced leftover inventory. We check robustness by verifying that multiplier adjusted prices changed smoothly across states.

We compared this approach to simpler variants such as using only the raw optimal price or using fixed segment prices. Simpler variants were either too rigid or ignored inventory. The combined optimal price times inventory multiplier provided the best balance.

Overall, we tested several variations, including global vs. segmented demand models, logistic vs. tree-based models, and pricing with or without inventory adjustments. Across all comparisons, the 8-segment XGBoost model combined with the inventory-pressure multiplier gave the strongest and most stable performance, so we selected this as our final approach.

Further Considerations (if Given More Time)

We notice that Covariate2 creates a large difference in demand between the high and low groups. If given more time, we would investigate the impact of Covariate2 on price gaps, as it influences both the segmentation and the purchase probability predictions. We would explore adjustments such as removing Covariate2 or reducing its correlation with other features to limit price gaps between groups while still preserving model performance.

Part 2: Pricing under competition

For part 2, our strategy is an Adaptive Strategy Selector (meta-agent) that dynamically deploys the best sub-agents (XGBoost Multiplier or DP Agent) in immediate response to opponent behavior.

Our meta agent initiates a Detection Phase during the first 80 steps of the sales cycle, collecting opponent historical quotes to calculate price variation and fluctuation frequency. If an opponent's price standard deviation is small and fluctuations are regular, it is identified as a Dynamic Pricing Agent. If an opponent's price fluctuates greatly and is frequently adjusted, it is identified as a Multiplier Agent.

To maximize total revenue, the meta-agent adapts its strategy. The system deploys Multiplier Agent against DP agent to avoid price wars and secure higher revenue (with slightly lower sales volume compared to DP agent). The system switches to DP Agent when against Multiplier competitors to ensure stable revenue and avoid being swayed by competitor volatility.

Strategy Evolution

We initially used Multiplier Agent (Ver. 1), an improved Part 1 XGBoost-based heuristic using pricing adjustment based on inventory pressure, time remaining and opponent's behaviour. This model performed well, achieving ~57K average revenue (AR). Following TA's suggestion, we developed two DP Agents (Ver. 1 & 2), but both underperformed yielding ~49K - 51K AR. This unexpected result led us to suspect that the problem wasn't the DP algorithm itself, but rather unintended "game interaction" (unhealthy competition) during specific matchups. To

verify this hypothesis, we decided to conduct a comprehensive cross-player simulation locally.

Simulation and Payoff Matrix

We conducted several full 500-move simulations between our Multiplier and DP Agents to derive a “payoff matrix” explaining the market dynamics under different strategy interactions.

- Multiplier vs. Multiplier: Yielded high profit (~10K -12K each)
- Multiplier vs. DP: DP Agent wins every time (~10K-11K) while the Multiplier Agent earns a considerable revenue (~7K- 8K).
- DP vs DP: Vicious price war with extremely low profits (~3K- 4K).

The DP Agent's low leaderboard performance was attributed to the prevalent DP-dominated environment. Our DP Agent's performance suffers because it frequently encounters other DP agents, leading to revenue-destroying price war. Crucially, the Multiplier Agent's losses (~7K+) far exceed the DP vs. DP price war revenue (~3K+), providing Multiplier agents perform better overall in a DP-heavy competitive environment.

Final Meta Agent Classification Mechanism

Our final Agent is an adaptive framework that identifies opponent behavior before selecting the optimal sub-agent, rather than committing to a single approach. Based on observations, we assumed opponents do not change agents mid-game and their price behavior reflects their strategy. Specifically, opponents with extremely low price fluctuations and small adjustments tend to use more rigid or DP strategies, while opponents with frequent and large price changes are more likely to use multiplier or reactive strategies.

The adaptive framework identifies the opponent's strategy within the first 80 steps by

calculating price stability indicators: price standard deviation (price_std) and small change ratio in adjacent prices (frac_small_move).

- Static/Predictable: If $\text{frac_small_move} > 0.7$ and $\text{price_std} < 15$, it switches to DP Agent.
 - Dynamic/Reactive: If fluctuations are large and frequent, it switches to Multiplier Agent
- Version 2. Once classified, the selected sub-agent is fixed for the remainder of the game.

Sub Agent Deployment and Strategy

The DP Agent is designed for opponents exhibiting stable, predictable price changes. Its strategy integrates four core components:

- Benchmark Price: Determines the buyer's segment and retrieves the optimal price from the offline DP policy, providing a stable baseline based on current inventory and time.
- Greedy Optimization: A vectorized method calculates expected revenue across 100 prices, fine-tuning the DP price to compensate for its conservative nature.
- Dynamic Multiplier: Adjusts the multiplier based on the recent sales success rate of each segment, allowing the price to dynamically follow market sentiment.
- Competitive Reaction: Reacts to the opponent's previous price by slightly lowering our price if we are higher. Continuous price suppression triggers an immediate conservative strategy to prevent price war.

The Multiplier Agent handles opponents with large price fluctuations and high reactivity. It combines XGBoost prediction with heuristic centered on a Dynamic Multiplier based on Market Saturation. This indicator combines remaining time pressure and total inventory of both agents. The system adjusts the quote in real time.

- High Supply: Multiplier is lowered to aggressively capture orders.

- High Demand: Multiplier is increased to retain inventory. This mechanism ensures optimal adaptation to inventory and time pressure in a dynamic competition.

LLM usage

We only used LLM for implementation tasks, not for creating our core strategy. It was used primarily for searching syntax, optimizing performance and debugging such as converting timing-out ‘for’ loops to fast NumPy vectorized operations. The team formulated all core pricing strategy and game logic first. All LLM-generated code was strictly reviewed and locally tested to ensure logic consistency.

CONCLUSION

We are satisfied with our final result, largely attributing our success to the Adaptive Strategy Selector used in the competitive phase. This agent’s core strength was its ability to accurately classify opponents as either static/predictable or dynamic/reactive, enabling the deployment of the optimal counter-strategy. However, the constantly changing competitive environment made it difficult to isolate the true performance of individual agent versions. For future improvement, more run time or strict, locked timeline simulations against a fixed set of opponents would be invaluable. As a group, we learned the critical lesson that algorithmic complexity is not always synonymous with optimal performance, underscoring the importance of thoroughly exploring the dataset and understanding the market environment before finalizing any strategy.